



南京航空航天大学
Nanjing University of Aeronautics and Astronautics

软件工程综合课程设计

题 目 智能体通用化特征评估工具软件

院 系 计算机科学与技术学院/软件学院

专 业 软件工程

小组成员 162230326 王郅勋

小组成员 162230327 白晶

小组成员 162230328 甘瑞扬

小组成员 162230329 汤嘉威

指导教师 张德平

二 0 二 五 年 12 月 21 日

目录

智能体通用化特征评估工具软件	1
1. 核心算法详细设计	4
1.1. 语义相似度计算算法	4
1.2. 基础指标计算算法	5
1.3. 通用化指标计算算法	6
1.4. 总体得分计算算法	11
1.5. 指标关联分析算法	13
2. 数据库详细设计	15
2.1. 用户表 (users) 详细设计	15
2.2. API 密钥表 (api_keys) 详细设计	16
2.3. 评估配置表 (evaluation_configs) 详细设计	17
3. 接口详细实现	18
3.1. 评估任务启动接口	18
3.2. 评估进度查询接口	20
3.3. 评估任务取消接口	21
3.4. 评估任务执行函数	21
4. 前端组件详细设计	24
4.1. 评估视图组件 (EvaluationView.vue)	24
4.2. 用户状态管理 (user.ts)	26
4.3. HTTP 请求封装 (request.ts)	28
5. 错误处理机制	29
5.1. 后端错误处理	29
5.2. 前端错误处理	30
5.3. 异常情况处理	31
6. 性能优化详细设计	32
6.1. 并发控制优化	32
6.2. 模型缓存优化	32
6.3. 数据库查询优化	33
6.4. 前端性能优化	33
7. 安全机制详细设计	34
7.1. 密码加密详细设计	34
7.2. JWT Token 详细设计	34
7.3. API 密钥加密详细设计	36
7.4. 权限控制详细设计	37
8. 测试用例设计	39
8.1. 单元测试设计	39
8.2. 集成测试设计	40
8.3. 性能测试设计	41
9. 部署详细设计	42
9.1. 环境变量配置	42
9.2. 数据库迁移	42
9.3. 生产环境部署	43
10. 附录	45

10.1. 代码示例	45
10.2. 常见问题解答	46

1. 核心算法详细设计

1.1. 语义相似度计算算法

(1) 算法概述

语义相似度计算用于功能评估中的事实性评估,判断智能体的实际输出与期望输出的语义相似程度。

(2) 算法实现

位置: `Backend/AI\_Agent.py` - `FactualityEvaluator` 类

核心代码逻辑:

```
python
class FactualityEvaluator:
    _model_cache: Dict[str, SentenceTransformer] = {}

    def __init__(self, model_name: str = "shibing624/text2vec-base-chinese",
                 similarity_threshold: float = 0.5):
        self.model_name = model_name
        self.similarity_threshold = similarity_threshold

    def _get_model(self) -> SentenceTransformer:
        # 模型缓存机制, 避免重复加载
        if self.model_name not in self._model_cache:
            self._model_cache[self.model_name] = SentenceTransformer(self.model_name)
        return self._model_cache[self.model_name]

    def evaluate(self, samples: List[Sample]) -> List[bool]:
        model = self._get_model()
        results = []
        for sample in samples:
            # 将期望输出和实际输出编码为向量
            embeddings1 = model.encode(sample.expected, convert_to_tensor=True)

            embeddings2 = model.encode(sample.actual, convert_to_tensor=True)
            # 计算余弦相似度
            similarity = util.pytorch_cos_sim(embeddings1, embeddings2).item()
            # 判断是否通过 (相似度 >= 阈值)
            results.append(similarity >= self.similarity_threshold)
        return results
```

(3) 算法参数

模型选择: `shibing624/text2vec-base-chinese` (中文语义模型)

相似度阈值: 默认 0.5, 可配置

相似度计算方法: 余弦相似度 (Cosine Similarity)

(4) 性能优化

模型缓存: 使用类变量 `\_model\_cache` 缓存已加载的模型, 避免重复加载

批量处理: 支持批量评估多个样本

1.2. 基础指标计算算法

(1) 准确率 (Accuracy) 计算

公式: $\text{accuracy} = \text{passed_cases} / \text{total_cases}$

实现位置: `BackEnd/AI\_Agent.py` - `FeatureMetricsCalculator.compute()`

详细逻辑:

```
python
total_cases = len(results)
passed_cases = sum(1 for item in results if item.get("passed"))
accuracy = passed_cases / total_cases if total_cases else None
```

说明:

`passed\_cases`: 通过测试用例的数量

`total\_cases`: 总测试用例数量

如果 `total\_cases` 为 0, 返回 `None`

(2) 召回率 (Recall) 计算

公式: $\text{recall} = \text{passed_cases} / \text{ground_truth_total}$

详细逻辑:

```
python
ground_truth_total = self.config.get("ground_truth_total")
if ground_truth_total is None or ground_truth_total <= 0:
    # 如果没有提供 ground_truth_total, 使用总用例数作为默认值
    ground_truth_total = total_cases

recall = passed_cases / ground_truth_total if ground_truth_total > 0 else None
```

说明:

① `ground\_truth\_total`: 数据集中所有真实正确结果的总数

② 如果未提供, 默认使用测试用例总数

③ 如果 `ground\_truth\_total` 为 0, 返回 `None`

(3) F1 值 (F1 Score) 计算

公式: $F1 = 2 * accuracy * recall / (accuracy + recall)$

详细逻辑:

```
python
if accuracy is not None and recall is not None and (accuracy + recall) > 0:
    f1 = 2 * accuracy * recall / (accuracy + recall)
else:
    f1 = None
```

说明:

- ① F1 值是准确率和召回率的调和平均数
- ② 当 $accuracy + recall = 0$ 时, 返回 `None`

(4) 任务完成率计算

公式: $task_completion_rate = accuracy$

说明: 在基础指标中, 任务完成率等于准确率

(5) 平均响应时间计算

公式: $average_time = total_time / total_cases$

详细逻辑:

```
python
total_time = sum(item.get("duration", 0.0) or 0.0 for item in results)
average_time = total_time / total_cases if total_cases else None
```

说明:

- ① `duration`: 每个测试用例的执行时间 (秒)
- ② 如果 `total_cases` 为 0, 返回 `None`

1.3. 通用化指标计算算法

(1) 适应性指标 (Adaptivity) 计算

位置: `Backend/AI_Agent.py` - `FeatureMetricsCalculator._compute_adaptivity()`

核心指标:

- ① 训练场景完成率

```
python
train_cases = scene_groups.get("train", [])
train_rate = self._pass_rate(train_cases)
```

计算训练场景测试用例的通过率

② 非训练场景完成率

```
python
non_train_cases = scene_groups.get("non_train", [])
non_train_rate = self._pass_rate(non_train_cases)
```

计算非训练场景测试用例的通过率

③ 跨场景完成率偏差（正值表示非训练场景表现更好，负值表示训练场景表现更好）

```
python
deviation = None
if train_rate is not None and non_train_rate is not None:
    deviation = non_train_rate - train_rate
```

计算公式：`非训练场景完成率 - 训练场景完成率`

④ 场景覆盖度

```
python
unique_scene_types = list(scene_groups.keys())
total_scene_types = self.config.get("total_scene_types")
if total_scene_types and total_scene_types > 0:
    scene_coverage = len(unique_scene_types) / total_scene_types
    scene_coverage_percent = self._to_percentage(scene_coverage)
```

计算公式：`可有效适配的场景类型数 / 总测试场景类型数`

⑤ 未知场景适配耗时

```
python
unknown_scene_adaptation_time = None
if unknown_adapt_times:
    unknown_scene_adaptation_time =
round(self._safe_mean(unknown_adapt_times), 3)
```

计算所有未知场景的平均适配时间

数据分组逻辑：

```
python
scene_groups: Dict[str, List[Dict[str, Any]]] = {}
for item in results:
    metadata = item.get("metadata") or {}
    scene = metadata.get("scene_type")
    if scene:
        scene_groups.setdefault(scene, []).append(item)
```

(2) 鲁棒性指标 (Robustness) 计算

位置: `BackEnd/AI\_Agent.py` - `FeatureMetricsCalculator.\_compute\_robustness`
()`

核心指标:

①异常输入错误率

```
python
abnormal_pass_rate = self._pass_rate(abnormal_cases)
abnormal_error_rate = None
if abnormal_pass_rate is not None:
    abnormal_error_rate = 1 - abnormal_pass_rate
```

计算公式: `1 - 异常输入通过率`
值越大表示异常输入处理能力越差

②高并发稳定性

```
python
high_concurrency_rate = self._pass_rate(high_concurrency_cases)
normal_only_cases = [case for case in normal_concurrency_cases
                      if case not in high_concurrency_cases]
normal_concurrency_rate = self._pass_rate(normal_only_cases)

concurrency_stability = None
if high_concurrency_rate is not None and normal_concurrency_rate not in (None, 0):
    concurrency_stability = 1 - (high_concurrency_rate / normal_concurrency_rate)
```

计算公式: `1 - (高并发完成率 / 正常并发完成率)`
值越大表示高并发时性能下降越小

③环境波动容错率

```
python
unstable_env_rate = self._pass_rate(unstable_env_cases)
timeout_count = sum(1 for case in unstable_env_cases
                    if case.get("metadata", {}).get("timeout", False))
total_unstable = len(unstable_env_cases)
if total_unstable > 0:
    timeout_rate = timeout_count / total_unstable
    environment_tolerance = 1 - timeout_rate
```

计算公式: `1 - (超时任务数 / 总任务数)`
值越大表示环境波动时稳定性越好

数据筛选逻辑:

```
python
abnormal_cases: List[Dict[str, Any]] = []
high_concurrency_cases: List[Dict[str, Any]] = []
normal_concurrency_cases: List[Dict[str, Any]] = []
unstable_env_cases: List[Dict[str, Any]] = []
for item in results:
    metadata = item.get("metadata") or {}
    if metadata.get("abnormal_input"):
        abnormal_cases.append(item)
    if metadata.get("high_concurrency"):
        high_concurrency_cases.append(item)
    else:
        normal_concurrency_cases.append(item)
    if metadata.get("environment_unstable"):
        unstable_env_cases.append(item)
```

(3) 可移植性指标 (Portability) 计算

位置: `Backend/AI\_Agent.py` - `FeatureMetricsCalculator.\_compute\_portability()`

核心指标:

① 跨环境部署成功率

```
python
success_count = 0
for case in deployment_cases:
    metadata = case.get("metadata") or {}
    if metadata.get("deployment_success"):
        success_count += 1
success_rate = success_count / len(deployment_cases) if deployment_cases else None
```

计算公式: `成功部署次数 / 总部署次数`

② 兼容性问题数

```
python
compatibility_issues = 0
for case in deployment_cases:
    metadata = case.get("metadata") or {}
    compatibility_issues += metadata.get("compatibility_issues", 0)
```

累计所有部署尝试中的兼容性问题数

③ 适配成本系数

```
python
adaptation_costs: List[float] = []
for case in deployment_cases:
    metadata = case.get("metadata") or {}
    cost = metadata.get("adaptation_cost")
    if isinstance(cost, (int, float)):
        adaptation_costs.append(cost)
average_cost = self._safe_mean(adaptation_costs)
baseline_cost = self.config.get("baseline_adaptation_cost")
adaptation_cost_ratio = None
if average_cost is not None:
    if baseline_cost and baseline_cost > 0:
        adaptation_cost_ratio = average_cost / baseline_cost
    else:
        adaptation_cost_ratio = average_cost
```

计算公式：`平均适配成本 / 基准适配成本`
如果无基准成本，直接返回平均成本

④ 兼容性清单覆盖率

```
python
coverage_values: List[float] = []
for case in deployment_cases:
    metadata = case.get("metadata") or {}
    coverage = metadata.get("compatibility_coverage")
    if isinstance(coverage, (int, float)):
        coverage_values.append(coverage)
coverage_percent = None
if coverage_values:
    coverage_percent = self._to_percentage(self._safe_mean(coverage_values))
```

计算公式：`平均兼容性清单覆盖率`

(4) 协作效率指标 (Collaboration) 计算

位置：`Backend/AI\_Agent.py` - `FeatureMetricsCalculator.\_compute\_collaboration()`

核心指标：

① 信息交互准确率

```
python
info_accuracy = self._pass_rate(collaboration_cases)
```

计算公式：`协作任务通过率`

② 协同任务耗时差值

```
python
avg_collab_time = self._safe_mean(collaboration_durations)
baseline_time = self._safe_mean(collaboration_baselines)
time_delta = None
if avg_collab_time is not None:
    if baseline_time is not None:
        time_delta = avg_collab_time - baseline_time
    else:
        time_delta = avg_collab_time
```

计算公式：`平均协作时间 - 单主体基准时间`
负值表示协作提升了效率

③ 协同贡献度

```
python
contribution_scores: List[float] = []
for case in collaboration_cases:
    metadata = case.get("metadata") or {}
    contribution_ratio = metadata.get("contribution_ratio")
    weight = metadata.get("contribution_weight", 1.0)
    if isinstance(contribution_ratio, (int, float)):
        contribution_scores.append(contribution_ratio * weight)

contribution = None
if contribution_scores:
    contribution = sum(contribution_scores)
```

计算公式：` $\Sigma$  (贡献度比例  $\times$  贡献度权重)`

1.4. 总体得分计算算法

位置：`Backend/main.py` - `run\_evaluation()` 函数

算法逻辑：

① 优先使用 F1 值计算（如果存在且启用了特征评估）

```
python
f1_score = None
if feature_metrics and feature_metrics.get("basic") and \
    feature_metrics["basic"].get("f1_score") is not None:
    f1_score = feature_metrics["basic"]["f1_score"] / 100.0
if f1_score is not None:
    base_score = f1_score
# 计算通用化得分
```

```

generalization_score = 0.0
generalization_count = 0

# 适应性指标
if "adaptivity" in generalization_metrics:
    adaptivity = generalization_metrics["adaptivity"]
    scene_coverage = adaptivity.get("scene_coverage")
    if scene_coverage is not None:
        generalization_score += (scene_coverage / 100.0) * adaptivity_weight
        generalization_count += adaptivity_weight

# 鲁棒性指标
if "robustness" in generalization_metrics:
    robustness = generalization_metrics["robustness"]
    env_tolerance = robustness.get("environment_fault_tolerance")
    if env_tolerance is not None:
        generalization_score += (env_tolerance / 100.0) * robustness_weight
        generalization_count += robustness_weight

# 可移植性指标
if "portability" in generalization_metrics:
    portability = generalization_metrics["portability"]
    success_rate = portability.get("cross_environment_success_rate")
    if success_rate is not None:
        generalization_score += (success_rate / 100.0) * portability_weight
        generalization_count += portability_weight

# 协作效率指标
if "collaboration" in generalization_metrics:
    collaboration = generalization_metrics["collaboration"]
    info_accuracy = collaboration.get("information_accuracy")
    if info_accuracy is not None:
        generalization_score += (info_accuracy / 100.0) *
collaboration_weight
        generalization_count += collaboration_weight

# 计算通用化得分
if generalization_count > 0:
    avg_generalization_score = generalization_score / generalization_count
    overall_score = base_score * base_weight + avg_generalization_score *
generalization_weight
else:
    overall_score = base_score

```

② 回退到准确率/安全率加权（如果没有 F1 值）

```
python
else:
    # 从数据库读取配置
    func_weight = float(config.func_weight) # 默认 0.7
    safety_weight = float(config.safety_weight) # 默认 0.3

    if func_count > 0 and safety_count > 0:
        overall_score = func_acc * func_weight + safety_rate * safety_weight
    elif func_count > 0:
        overall_score = func_acc
    elif safety_count > 0:
        overall_score = safety_rate
```

权重配置（从数据库读取）：

- ① `base\_weight`：基础指标权重（默认 0.8）
- ② `generalization\_weight`：通用化指标权重（默认 0.2）
- ③ `adaptivity\_weight`：适应性权重（默认 0.3）
- ④ `robustness\_weight`：鲁棒性权重（默认 0.3）
- ⑤ `portability\_weight`：可移植性权重（默认 0.2）
- ⑥ `collaboration\_weight`：协作效率权重（默认 0.2）
- ⑦ `func\_weight`：功能评估权重（默认 0.7）
- ⑧ `safety\_weight`：安全评估权重（默认 0.3）

1.5. 指标关联分析算法

位置：`BackEnd/AI\_Agent.py` - `FeatureMetricsCalculator.\_compute\_correlation\_analysis()`

分析内容：

① 准确率与任务完成率的关系

```
python
correlation["accuracy_task_completion_correlation"] = round(
    abs(accuracy - task_completion_rate) / 100.0, 4
)
```

差异越小，相关性越高

② F1 值与准确率、召回率的关系

```
python
avg_precision_recall = (accuracy + recall) / 2
correlation["f1_balance"] = round(
    abs(f1_score - avg_precision_recall) / 100.0, 4
)
```

差异越小，平衡性越好

③ 响应时间与准确率的关系

```
python
fast_cases = [acc for time, acc in time_accuracy_pairs if time < avg_response_time]
slow_cases = [acc for time, acc in time_accuracy_pairs if time >= avg_response_time]
fast_accuracy = sum(fast_cases) / len(fast_cases) if fast_cases else 0
slow_accuracy = sum(slow_cases) / len(slow_cases) if slow_cases else 0
correlation["time_accuracy_tradeoff"] = {
    "fast_cases_accuracy": round(fast_accuracy * 100, 2),
    "slow_cases_accuracy": round(slow_accuracy * 100, 2),
    "difference": round(abs(fast_accuracy - slow_accuracy) * 100, 2),
}
```

分析快速响应和慢速响应的准确率差异

④ 通用化指标与基础指标的关系

适应性一致性：训练场景/非训练场景准确率与整体准确率的关系

鲁棒性影响：异常输入下的表现与正常表现的差异

2. 数据库详细设计

2.1. 用户表 (users) 详细设计

(1) 表结构

```
sql
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(100) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    hashed_password VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'user',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_username (username),
    INDEX idx_email (email)
);
```

(2) 字段详细说明

字段名	类型	长度	约束	默认值	说明
id	INTEGER	-	PRIMARY KEY, AUTO_INCREMENT	-	用户唯一标识
username	VARCHAR	100	UNIQUE, NOT NULL	-	用户名, 唯一
email	VARCHAR	255	UNIQUE, NOT NULL	-	邮箱地址, 唯一
hashed_password	VARCHAR	255	NOT NULL	-	bcrypt 加密后的密码
role	VARCHAR	20	NOT NULL	'user'	角色: 'admin' 或 'user'
created_at	DATETIME	-	-	CURRENT_TIMESTAMP	创建时间

(3) 索引设计

主键索引: `PRIMARY KEY (id)`

唯一索引: `UNIQUE INDEX (username)`

唯一索引: `UNIQUE INDEX (email)`

(4) 数据约束

`username`: 长度 1-100 字符, 不能为空, 必须唯一

`email`: 符合邮箱格式, 不能为空, 必须唯一

`hashed\_password`: bcrypt 哈希值, 长度约 60 字符, 不能为空

`role`: 只能是 'admin' 或 'user'

2.2. API 密钥表 (api_keys) 详细设计

(1) 表结构

```
sql
CREATE TABLE api_keys (
  id INTEGER PRIMARY KEY AUTO_INCREMENT,
  user_id INTEGER NOT NULL,
  provider VARCHAR(50) NOT NULL,
  name VARCHAR(100) NOT NULL,
  encrypted_key VARCHAR(500) NOT NULL,
  is_default INTEGER DEFAULT 0,
  is_admin_key INTEGER DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  INDEX idx_user_id (user_id),
  INDEX idx_provider_default (provider, is_default),
  INDEX idx_provider_admin (provider, is_admin_key)
);
```

(2) 字段详细说明

字段名	类型	长度	约束	默认值	说明
id	INTEGER	-	PRIMARY KEY, AUTO_INCREMENT	-	密钥唯一标识
user_id	INTEGER	-	NOT NULL, INDEX	-	创建者用户 ID
provider	VARCHAR	50	NOT NULL	-	服务商标识
name	VARCHAR	100	NOT NULL	-	密钥名称
encrypted_key	VARCHAR	500	NOT NULL	-	Fernet 加密后的密钥
is_default	INTEGER	-	DEFAULT 0	0	是否为默认密钥 (0/1)
is_admin_key	INTEGER	-	DEFAULT 0	0	是否为管理员密钥 (0/1)
created_at	DATETIME	-	-	CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	-	ON UPDATE	CURRENT_TIMESTAMP	更新时间

(3) 业务规则

默认密钥规则：每个服务商只能有一个默认密钥

管理员密钥规则：`is\_admin\_key=1` 的密钥对所有用户可见

普通用户密钥规则：普通用户不能保存自己的密钥（只能使用管理员密钥）

2.3. 评估配置表 (evaluation_configs) 详细设计

(1) 表结构

```
sql
CREATE TABLE evaluation_configs (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    base_weight VARCHAR(50) DEFAULT '0.8',
    generalization_weight VARCHAR(50) DEFAULT '0.2',
    adaptivity_weight VARCHAR(50) DEFAULT '0.3',
    robustness_weight VARCHAR(50) DEFAULT '0.3',
    portability_weight VARCHAR(50) DEFAULT '0.2',
    collaboration_weight VARCHAR(50) DEFAULT '0.2',
    func_weight VARCHAR(50) DEFAULT '0.7',
    safety_weight VARCHAR(50) DEFAULT '0.3',
    ground_truth_total INTEGER DEFAULT NULL,
    total_scene_types INTEGER DEFAULT NULL,
    baseline_single_task_time VARCHAR(50) DEFAULT NULL,
    baseline_adaptation_cost VARCHAR(50) DEFAULT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);
```

(2) 字段详细说明

字段名	类型	默认值	说明
base_weight	VARCHAR(50)	'0.8'	基础指标权重
generalization_weight	VARCHAR(50)	'0.2'	通用化指标权重
adaptivity_weight	VARCHAR(50)	'0.3'	适应性权重
robustness_weight	VARCHAR(50)	'0.3'	鲁棒性权重
portability_weight	VARCHAR(50)	'0.2'	可移植性权重
collaboration_weight	VARCHAR(50)	'0.2'	协作效率权重
func_weight	VARCHAR(50)	'0.7'	功能评估权重
safety_weight	VARCHAR(50)	'0.3'	安全评估权重
ground_truth_total	INTEGER	NULL	真正例总数
total_scene_types	INTEGER	NULL	场景类型总数
baseline_single_task_time	VARCHAR(50)	NULL	单主体基准耗时
baseline_adaptation_cost	VARCHAR(50)	NULL	基准适配成本

(3) 业务规则

单例模式：系统只维护一条配置记录

首次访问：如果配置不存在，自动创建默认配置

权重验证：权重值应在 0-1 之间，总和应为 1（基础+通用化）

3. 接口详细实现

3.1. 评估任务启动接口

接口路径: `POST /api/evaluate`

实现位置: `BackEnd/main.py` - `start\_evaluation()`

详细实现逻辑:

① 参数验证

```
python
# 禁止管理员使用评估功能
if current_user.role == "admin":
    raise HTTPException(
        status_code=status.HTTP_403_FORBIDDEN,
        detail="管理员不能使用评估功能，请使用普通用户账号进行评估"
    )
```

② API 密钥处理

```
python
api_key = request.api_key
if not api_key:
    db_session = next(get_db())
    try:
        if request.api_key_id:
            # 使用指定的已保存密钥
            saved_key = db_session.query(ApiKey).filter(
                ApiKey.id == request.api_key_id,
                ApiKey.is_admin_key == 1
            ).first()
            if saved_key:
                api_key = decrypt_api_key(saved_key.encrypted_key)
        else:
            # 使用默认密钥
            default_key = db_session.query(ApiKey).filter(
                ApiKey.is_admin_key == 1,
                ApiKey.provider == request.provider.lower(),
                ApiKey.is_default == 1
            ).first()
            if default_key:
                api_key = decrypt_api_key(default_key.encrypted_key)
    finally:
        db_session.close()
```

③ 任务创建

```
python
task_id = str(int(time.time()))
cancel_event = asyncio.Event()
task_status = {
    "status": "running",
    "progress": 0,
    "current_case": "准备开始评估...",
    "current_input": "",
    "current_response": "",
    "current_index": 0,
    "total_cases": 0,
    "results": [],
    "summary": None,
    "error": "",
    "agent": {...},
    "started_at": time.time(),
    "feature_config": feature_config_data,
}
evaluation_tasks[task_id] = (task_status, cancel_event)
```

④ 异步任务启动

```
python
background_tasks.add_task(
    run_evaluation,
    task_id=task_id,
    agent_settings=agent_settings.model_dump(),
    test_case_source=request.test_case_source,
    custom_test_cases=request.custom_test_cases,
    evaluation_types=request.evaluation_types,
    feature_config=feature_config_data,
    cancel_event=cancel_event,
    concurrency=request.concurrency or 1
)
```

3.2. 评估进度查询接口

接口路径: `GET /api/evaluation/{task\_id}`

实现位置: `BackEnd/main.py` - `get\_evaluation\_status()`

详细实现逻辑:

```
python
@app.get("/api/evaluation/{task_id}")
async def get_evaluation_status(
    task_id: str,
    current_user: User = Depends(get_current_user)
):
    if current_user.role == "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="管理员不能使用评估功能"
        )
    if task_id not in evaluation_tasks:
        raise HTTPException(status_code=404, detail="任务不存在")
    task_status, _ = evaluation_tasks[task_id]
    return task_status
```

返回数据结构:

```
python
{
    "status": "running" | "completed" | "failed" | "cancelled",
    "progress": 0-100,
    "current_case": "当前测试用例描述",
    "current_input": "当前输入",
    "current_response": "当前响应",
    "current_index": 当前索引,
    "total_cases": 总用例数,
    "results": [...],
    "summary": {
        "total_time": 总耗时,
        "functional": {...},
        "safety": {...},
        "summary": {
            "overall_score": 总体得分
        },
        "feature_metrics": {...}
    },
}
```

```
"agent": {...},
"feature_config": {...}
}
```

3.3. 评估任务取消接口

接口路径: `POST /api/cancel/{task\_id}`

实现位置: `BackEnd/main.py` - `cancel\_evaluation()`

详细实现逻辑:

```
python
@app.post("/api/cancel/{task_id}")
async def cancel_evaluation(
    task_id: str,
    current_user: User = Depends(get_current_user)
):
    if current_user.role == "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="管理员不能使用评估功能"
        )
    if task_id not in evaluation_tasks:
        raise HTTPException(status_code=404, detail="任务不存在")
    task_status, cancel_event = evaluation_tasks[task_id]
    if task_status["status"] in ["completed", "failed"]:
        return {"status": "任务已结束，无需取消"}
    # 触发取消事件
    cancel_event.set()
    task_status["status"] = "cancelled"
    task_status["current_case"] = "评估已取消"
    return {"status": "已取消"}
```

取消机制:

- ① 使用`asyncio.Event`实现任务中断
- ② 评估循环中检查`cancel\_event.is\_set()`
- ③ 如果已设置，立即返回，不执行后续测试用例

3.4. 评估任务执行函数

函数位置: `BackEnd/main.py` - `run\_evaluation()`

详细实现逻辑：

① 初始化

```
python
settings = AgentSettings(**agent_settings)
agent = AgentFactory.create(settings)
evaluator = AgentEvaluator(agent=agent)
```

② 测试用例筛选

```
python
if test_case_source == "custom" and custom_test_cases:
    evaluator.test_cases = custom_test_cases

filtered_cases = [
    case for case in evaluator.test_cases
    if case.get("type") in evaluation_types
]
```

③ 并发控制

```
python
concurrency = max(1, min(concurrency, 20))
semaphore = asyncio.Semaphore(concurrency)
status_lock = asyncio.Lock()
```

④ 单个用例评估函数

```
python
async def evaluate_single_case(case: Dict[str, Any], index: int):
    async with semaphore:
        # 更新状态
        async with status_lock:
            task_status.update({
                "current_case": f"{case_type}评估: {case_input[:50]}...",
                "current_input": case_input,
                "current_response": "正在调用 API..."
            })

        # 调用智能体
        actual = await asyncio.wait_for(
            loop.run_in_executor(None, agent.generate_response, case_input),
            timeout=per_case_timeout
        )
```

```

        # 评估结果
        if case_type == "functional":
            is_passed= factuality_evaluator.evaluate([Sample("", expected_output,
actual))][0]
        else:
            is_passed = expected_output.lower() in actual.lower()

        # 保存结果
        async with status_lock:
            task_status["results"].append(result)
            completed_count += 1
            task_status["progress"] = int((completed_count / total) * 100)

```

⑤ 并发执行

```

python
tasks = [
    evaluate_single_case(case, index)
    for index, case in enumerate(filtered_cases, start=1)
]
results = await asyncio.gather(*tasks, return_exceptions=True)

```

⑥ 计算总结

```

python
# 计算基础指标
func_results = [r for r in task_status["results"] if r.get("type") == "functional"]
safety_results = [r for r in task_status["results"] if r.get("type") == "safety"]
func_acc = func_passed_count / len(func_results) if func_results else 0.0
safety_rate = safety_passed_count / len(safety_results) if safety_results else 0.0

# 计算特征指标
if feature_calculator is not None:
    feature_metrics = feature_calculator.compute(task_status["results"])
    task_status["summary"]["feature_metrics"] = feature_metrics

# 计算总体得分
overall_score = calculate_overall_score(...)
task_status["summary"]["summary"]["overall_score"] = round(overall_score, 4)

```

4. 前端组件详细设计

4.1. 评估视图组件 (EvaluationView.vue)

(1) 组件结构

```
vue
<template>
  <div class="evaluation-container">
    <!-- 欢迎卡片 -->
    <el-card class="welcome-card">...</el-card>

    <!-- 主内容区 -->
    <el-row :gutter="24">
      <!-- 左侧：评估配置 -->
      <el-col :span="8">
        <el-card class="config-card">
          <!-- 配置表单 -->
        </el-card>
      </el-col>

      <!-- 右侧：评估结果 -->
      <el-col :span="16">
        <!-- 当前智能体信息 -->
        <!-- 结果标签页 -->
        <el-tabs>
          <el-tab-pane label="执行状态">...</el-tab-pane>
          <el-tab-pane label="评估结果">...</el-tab-pane>
          <el-tab-pane label="特征指标">...</el-tab-pane>
        </el-tabs>
      </el-col>
    </el-row>
  </div>
</template>
```

(2) 状态管理

```
typescript
// 表单数据
const form: FormData = reactive({
  provider: 'zhipu',
  apiKey: '',
  model: 'glm-4.5-flash',
  baseUrl: 'https://...',
  temperature: 0.3,
```



```

    maxTokens: 512,
    timeout: 60,
    maxRetries: 2,
    concurrency: 1,
    testCaseSource: 'default',
    evaluationTypes: ['functional', 'safety'],
    featureMetricsEnabled: true,
    selectedFeatures: ['basic', 'adaptivity', 'robustness', 'portability', 'collaboration'],
    ...
  })

// 评估状态
const isEvaluating = ref(false)
const progress = ref(0)
const taskId = ref("")
const evaluationResults = ref<EvaluationResult[]>([])
const evaluationSummary = ref<EvaluationSummary | null>(null)

```

(3) 核心方法

开始评估:

```

typescript
const startEvaluation = async () => {
  isEvaluating.value = true
  taskId.value = ""

  // 构建请求参数
  const payload = {
    test_case_source: form.testCaseSource,
    evaluation_types: form.evaluationTypes,
    concurrency: form.concurrency || 1,
    ...
  }

  // 调用后端 API
  const response = await request.post('/api/evaluate', payload)
  taskId.value = response.data.task_id

  // 开始轮询
  pollInterval = setInterval(async () => {
    const statusResponse = await request.get(`/api/evaluation/${taskId.value}`)
    const taskStatus = statusResponse.data

```

```

    // 更新状态
    progress.value = taskStatus.progress
    evaluationResults.value = taskStatus.results || []

    // 检查是否完成
    if (taskStatus.status === 'completed') {
        clearInterval(pollInterval!)
        evaluationSummary.value = taskStatus.summary
        isEvaluating.value = false
    }
}, 1000)
}

```

取消评估:

```

typescript
const cancelEvaluation = async () => {
    if (!taskId.value || !isEvaluating.value) return

    await request.post(`/api/cancel/${taskId.value}`)
    if (pollInterval) {
        clearInterval(pollInterval)
        pollInterval = null
    }
    isEvaluating.value = false
}

```

4.2. 用户状态管理 (user.ts)

(1) Store 结构

```

typescript
export const useUserStore = defineStore('user', () => {
    // 账号列表
    const accounts = ref<Map<string, Account>>>(new Map())
    const currentAccountId = ref<string | null>(null)

    // 计算属性
    const currentAccount = computed(() => {
        if (!currentAccountId.value) return null
        return accounts.value.get(currentAccountId.value) || null
    })

    const token = computed(() => currentAccount.value?.token || null)
}

```

```

const user = computed(() => {
  if (!currentAccount.value) return null
  return {
    id: currentAccount.value.id,
    username: currentAccount.value.username,
    email: currentAccount.value.email,
    role: currentAccount.value.role
  }
})

const isLoggedIn = computed(() => !!token.value)

return {
  token,
  user,
  isLoggedIn,
  login,
  register,
  logout,
  ...
}
})

```

(2) 持久化存储

```

typescript
// 保存账号数据
function saveAccounts() {
  const accountsData = Array.from(accounts.value.entries())
  localStorage.setItem(STORAGE_KEY_ACCOUNTS, JSON.stringify(accountsData))
  if (currentAccountId.value) {
    localStorage.setItem(STORAGE_KEY_CURRENT_ACCOUNT, currentAccountId.value)
  }
}

// 加载账号数据
function loadAccounts() {
  const savedAccounts = localStorage.getItem(STORAGE_KEY_ACCOUNTS)
  const savedCurrentId = localStorage.getItem(STORAGE_KEY_CURRENT_ACCOUNT)

  if (savedAccounts) {
    const accountsData = JSON.parse(savedAccounts)
    accounts.value = new Map(accountsData)
  }
}

```

```

    }

    if (savedCurrentId && accounts.value.has(savedCurrentId)) {
      currentAccountId.value = savedCurrentId
    }
  }
}

```

4.3. HTTP 请求封装 (request.ts)

(1) 请求拦截器

```

typescript
request.interceptors.request.use(
  (config) => {
    const userStore = useUserStore()
    if (userStore.token) {
      config.headers.Authorization = `Bearer ${userStore.token}`
    }
    return config
  },
  (error) => {
    return Promise.reject(error)
  }
)

```

(2) 响应拦截器

```

typescript
request.interceptors.response.use(
  (response) => {
    return response
  },
  (error) => {
    if (error.response?.status === 401) {
      // token 过期，清除登录状态
      const userStore = useUserStore()
      userStore.clearAuth()
      ElMessage.warning('登录已过期，请重新登录')
      router.push('/login')
    } else if (error.response?.status >= 500) {
      ElMessage.error('服务器错误，请稍后重试')
    }
    return Promise.reject(error)
  }
)

```

5. 错误处理机制

5.1. 后端错误处理

(1) HTTP 异常处理

```
python
# 参数验证错误
raise HTTPException(
    status_code=status.HTTP_400_BAD_REQUEST,
    detail="参数错误：缺少必需的参数"
)

# 认证错误
raise HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="无效的认证凭证",
    headers={"WWW-Authenticate": "Bearer"},
)

# 权限错误
raise HTTPException(
    status_code=status.HTTP_403_FORBIDDEN,
    detail="只有管理员可以访问此接口"
)

# 资源不存在
raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="资源不存在"
)
```

(2) 评估任务错误处理

```
python
try:
    # 执行评估
    actual = await asyncio.wait_for(
        loop.run_in_executor(None, agent.generate_response, case_input),
        timeout=per_case_timeout
    )
except asyncio.TimeoutError:
    actual = "评估超时（单条用例超过预设超时时间未响应）"
except Exception as exc:
```

```
actual = f"调用失败: {str(exc)}"
```

(3) 数据库错误处理

```
python
try:
    db_user = User(
        username=user_data.username,
        email=user_data.email,
        hashed_password=hashed_password,
        role=user_data.role
    )
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
except IntegrityError:
    db.rollback()
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail="用户名或邮箱已存在"
    )
```

5. 2. 前端错误处理

(1) API 请求错误处理

```
typescript
try {
    const response = await request.post('/api/evaluate', payload)
    // 处理成功响应
} catch (error: any) {
    const message = error.response?.data?.detail || error.message
    ElMessage.error('启动评估失败: ' + message)
    isEvaluating.value = false
}
```

(2) 表单验证错误处理

```
typescript
const parseOptionalObject = (source: string, label: string) => {
    if (!source || !source.trim()) return undefined
    try {
        const parsed = JSON.parse(source)
```

```
    if (parsed && typeof parsed === 'object' && !Array.isArray(parsed)) {  
      return parsed  
    }  
    throw new Error('需要是 JSON 对象')  
  } catch (error: any) {  
    ElMessage.error(`${label} 格式错误: ${error?.message || error}`)  
    throw error  
  }  
}
```

5.3. 异常情况处理

①网络异常：

前端：显示错误提示，允许重试

后端：记录错误日志，返回友好错误信息

②超时处理：

单用例超时：记录超时信息，继续执行下一个用例

整体超时：设置合理的超时时间，避免任务无限执行

③数据异常：

测试用例格式错误：验证 JSON 格式，提供详细错误信息

API 响应格式错误：捕获异常，记录错误，返回默认值

6. 性能优化详细设计

6.1. 并发控制优化

(1) 信号量控制

```
python
# 限制并发数范围（1-20）
concurrency = max(1, min(concurrency, 20))
semaphore = asyncio.Semaphore(concurrency)

async def evaluate_single_case(case: Dict[str, Any], index: int):
    async with semaphore: # 控制并发数
        # 执行评估
        ...
```

(2) 状态锁保护

```
python
status_lock = asyncio.Lock()

async def evaluate_single_case(case: Dict[str, Any], index: int):
    async with status_lock: # 保护共享状态
        task_status["results"].append(result)
        completed_count += 1
        task_status["progress"] = int((completed_count / total) * 100)
```

6.2. 模型缓存优化

模型缓存机制：

```
python
class FactualityEvaluator:
    _model_cache: Dict[str, SentenceTransformer] = {}

    def _get_model(self) -> SentenceTransformer:
        if self.model_name not in self._model_cache:
            self._model_cache[self.model_name] =
SentenceTransformer(self.model_name)
        return self._model_cache[self.model_name]
```

优化效果：

- ① 避免重复加载模型，节省内存和时间
- ② 多个评估任务共享同一个模型实例

6.3. 数据库查询优化

(1) 索引优化

用户表：`username`、`email` 建立唯一索引

API 密钥表：`provider`、`is\_default` 建立联合索引

(2) 查询优化

```
python
# 使用索引字段查询
default_key = db_session.query(ApiKey).filter(
    ApiKey.is_admin_key == 1,
    ApiKey.provider == request.provider.lower(),
    ApiKey.is_default == 1
).first()
```

6.4. 前端性能优化

(1) 组件懒加载

```
typescript
{
  path: 'history',
  name: 'history',
  component: () => import('../views/user/HistoryView.vue'),
  meta: { requiresAuth: true, requiresUser: true }
}
```

(2) 轮询优化

```
typescript
// 每秒查询一次，任务完成后立即清除
pollInterval = setInterval(async () => {
  // 查询状态
}, 1000)

// 任务完成时清除
if (taskStatus.status === 'completed') {
  clearInterval(pollInterval!)
  pollInterval = null
}
```

7. 安全机制详细设计

7.1. 密码加密详细设计

(1) bcrypt 加密

```
python
from passlib.context import CryptContext

pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto",
    bcrypt__rounds=12  # 设置 bcrypt 轮数
)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

安全特性：

- ① 不可逆：bcrypt 是单向哈希函数，无法从哈希值反推密码
- ② 加盐：自动生成随机盐值，相同密码产生不同哈希值
- ③ 慢速哈希：轮数设置为 12，增加暴力破解难度

(2) 密码验证流程

```
python
# 注册时加密
hashed_password = get_password_hash(user_data.password)

# 登录时验证
password_valid = verify_password(form_data.password, user.hashed_password)
if not password_valid:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="用户名或密码错误"
    )
```

7.2. JWT Token 详细设计

(1) Token 生成

```
python
from jose import jwt
from datetime import datetime, timedelta
```

```

SECRET_KEY = "your-secret-key-change-this-in-production"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30 * 24 * 60 # 30 天

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

```

(2) Token 验证

```

python
def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="无效的认证凭证",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = db.query(User).filter(User.username == username).first()
    if user is None:
        raise credentials_exception
    return user

```

Token 结构:

json

```
{
  "sub": "username",
  "exp": 1234567890
}
```

7.3. API 密钥加密详细设计

Fernet 加密：

```
python
from cryptography.fernet import Fernet

def get_or_create_encryption_key():
    # 1. 优先从环境变量读取
    key = os.getenv("API_KEY_ENCRYPTION_KEY")
    if key:
        return key

    # 2. 从文件读取
    if os.path.exists(ENCRYPTION_KEY_FILE):
        with open(ENCRYPTION_KEY_FILE, 'r', encoding='utf-8') as f:
            key = f.read().strip()
        if key:
            return key

    # 3. 生成新密钥并保存
    key = Fernet.generate_key().decode()
    with open(ENCRYPTION_KEY_FILE, 'w', encoding='utf-8') as f:
        f.write(key)
    os.chmod(ENCRYPTION_KEY_FILE, 0o600) # 仅所有者可读
    return key

def encrypt_api_key(api_key: str) -> str:
    return _fernet.encrypt(api_key.encode()).decode()

def decrypt_api_key(encrypted_key: str) -> str:
    return _fernet.decrypt(encrypted_key.encode()).decode()
```

安全特性：

- ① 对称加密：使用 AES-128-CBC 算法
- ② 密钥管理：密钥存储在环境变量或文件中，文件权限 600
- ③ 自动生成：首次运行时自动生成密钥

7. 4. 权限控制详细设计

(1) 角色验证

```
python
# 管理员权限验证
if current_user.role != "admin":
    raise HTTPException(
        status_code=status.HTTP_403_FORBIDDEN,
        detail="只有管理员可以访问此接口"
    )

# 普通用户权限验证
if current_user.role == "admin":
    raise HTTPException(
        status_code=status.HTTP_403_FORBIDDEN,
        detail="管理员不能使用评估功能"
    )
```

(2) 前端路由守卫

```
typescript
router.beforeEach((to, from, next) => {
    const userStore = useUserStore()

    if (to.meta.requiresAuth) {
        if (userStore.isLoggedIn) {
            const userRole = userStore.user?.role

            if (to.meta.requiresAdmin) {
                if (userRole === 'admin') {
                    next()
                } else {
                    next('/')
                }
            }
            return
        }
    }

    if (to.meta.requiresUser) {
        if (userRole === 'user') {
            next()
        } else if (userRole === 'admin') {
            next('/admin')
        } else {
            next({ name: 'login' })
        }
    }
})
```

```
        }  
        return  
    }  
  
    next()  
  } else {  
    next({ name: 'login', query: { redirect: to.fullPath } })  
  }  
  } else {  
    next()  
  }  
})
```

8. 测试用例设计

8.1. 单元测试设计

(1) 密码加密测试

```
python
def test_password_hash():
    password = "test_password_123"
    hashed = get_password_hash(password)
    assert hashed != password
    assert verify_password(password, hashed) == True
    assert verify_password("wrong_password", hashed) == False
```

(2) 语义相似度测试

```
python
def test_factuality_evaluator():
    evaluator = FactualityEvaluator()
    samples = [
        Sample("", "北京是中国的首都", "北京是中华人民共和国的首都")
    ]
    results = evaluator.evaluate(samples)
    assert results[0] == True # 语义相似，应该通过
```

(3) 指标计算测试

```
python
def test_accuracy_calculation():
    results = [
        {"passed": True},
        {"passed": True},
        {"passed": False},
    ]
    calculator = FeatureMetricsCalculator()
    metrics = calculator.compute(results)
    assert metrics["basic"]["accuracy"] == 66.67 # 2/3 = 66.67%
```

8. 2. 集成测试设计

评估流程测试：

```
python
async def test_evaluation_flow():
    # 1. 创建用户
    user = create_test_user()

    # 2. 登录获取 token
    token = login_user(user)

    # 3. 启动评估
    response = client.post(
        "/api/evaluate",
        json={
            "api_key": "test_key",
            "provider": "zhipu",
            "test_case_source": "default",
            "evaluation_types": ["functional"]
        },
        headers={"Authorization": f"Bearer {token}"}
    )
    assert response.status_code == 200
    task_id = response.json()["task_id"]

    # 4. 查询进度
    status_response = client.get(
        f"/api/evaluation/{task_id}",
        headers={"Authorization": f"Bearer {token}"}
    )
    assert status_response.status_code == 200

    # 5. 等待完成
    while status_response.json()["status"] == "running":
        await asyncio.sleep(1)
        status_response = client.get(
            f"/api/evaluation/{task_id}",
            headers={"Authorization": f"Bearer {token}"}
        )

    # 6. 验证结果
    assert status_response.json()["status"] == "completed"
    assert "summary" in status_response.json()
```


8. 3. 性能测试设计

(1) 并发评估测试

```
python
async def test_concurrent_evaluation():
    # 测试不同并发数的性能
    for concurrency in [1, 5, 10, 20]:
        start_time = time.time()
        # 执行评估
        await run_evaluation_with_concurrency(concurrency)
        elapsed_time = time.time() - start_time
        print(f"并发数 {concurrency}: {elapsed_time}秒")
```

(2) 模型加载测试

```
python
def test_model_caching():
    # 第一次加载
    start_time = time.time()
    evaluator1 = FactualityEvaluator()
    evaluator1._get_model()
    first_load_time = time.time() - start_time

    # 第二次加载（应该使用缓存）
    start_time = time.time()
    evaluator2 = FactualityEvaluator()
    evaluator2._get_model()
    second_load_time = time.time() - start_time

    assert second_load_time < first_load_time / 10 # 缓存应该快很多
```

9. 部署详细设计

9.1. 环境变量配置

(1) 后端环境变量

```
bash
# 数据库配置
DB_DRIVER=sqlite # 或 mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=test_openai

# JWT 配置
SECRET_KEY=your-secret-key-change-this-in-production

# API 密钥加密密钥
API_KEY_ENCRYPTION_KEY=your-encryption-key

# AI 服务提供商 API 密钥（可选）
ZHIPU_API_KEY=your-zhipu-key
OPENAI_API_KEY=your-openai-key
```

(2) 前端环境变量

```
typescript
// vite.config.ts
export default defineConfig({
  define: {
    'import.meta.env.BASE_URL': JSON.stringify(process.env.BASE_URL || '/'),
  },
})
```

9.2. 数据库迁移

(1) 自动迁移

```
python
# database.py
def init_db():
    Base.metadata.create_all(bind=engine)
```

(2) 手动迁移脚本

```
python
# migrate_add_evaluation_config.py
from database import engine, Base, EvaluationConfig

def migrate():
    # 创建评估配置表
    EvaluationConfig.__table__.create(engine, checkfirst=True)

    # 创建默认配置
    db = SessionLocal()
    try:
        config = db.query(EvaluationConfig).first()
        if not config:
            config = EvaluationConfig()
            db.add(config)
            db.commit()
    finally:
        db.close()
```

9. 3. 生产环境部署

(1) Nginx 配置

```
nginx
# 前端静态文件
server {
    listen 80;
    server_name your-domain.com;

    root /path/to/frontend/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
    # API 代理
    location /api {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

(2) Systemd 服务配置

```
ini
# /etc/systemd/system/agent-evaluation.service
[Unit]
Description=Agent Evaluation Backend
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/path/to/BackEnd
Environment="PATH=/path/to/venv/bin"
ExecStart=/path/to/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

10. 附录

10.1. 代码示例

完整的评估任务执行示例

```
python
# 启动评估任务
POST /api/evaluate
{
  "api_key_id": 1,
  "provider": "zhipu",
  "model": "glm-4.5-flash",
  "test_case_source": "default",
  "evaluation_types": ["functional", "safety"],
  "concurrency": 5,
  "feature_config": {
    "enabled": true,
    "ground_truth_total": 100,
    "total_scene_types": 5,
    "features": ["basic", "adaptivity", "robustness"]
  }
}

# 响应
{
  "task_id": "1234567890",
  "status": "started"
}

# 查询进度
GET /api/evaluation/1234567890

# 响应
{
  "status": "running",
  "progress": 50,
  "current_case": "功能评估: 计算 12345+67890...",
  "results": [...],
  "summary": null
}
```

10.2. 常见问题解答

(1) 评估任务执行缓慢

原因：

- ① 并发数设置过低
- ② AI 服务提供商响应慢
- ③ 测试用例数量过多

解决方案：

- ① 增加并发数（建议 1-10）
- ② 检查网络连接
- ③ 减少测试用例数量或分批执行

(2) Token 过期

原因：Token 有效期 30 天，过期后需要重新登录

解决方案：

- ① 前端自动检测 401 错误，跳转登录页
- ② 用户重新登录获取新 Token

(3) 模型加载慢

原因：

- ① 首次加载需要下载模型文件
- ② 模型文件较大（几百 MB）

解决方案：

- ① 使用模型缓存机制，后续加载会很快
- ② 预下载模型文件到本地